

LAPORAN PRAKTIKUM

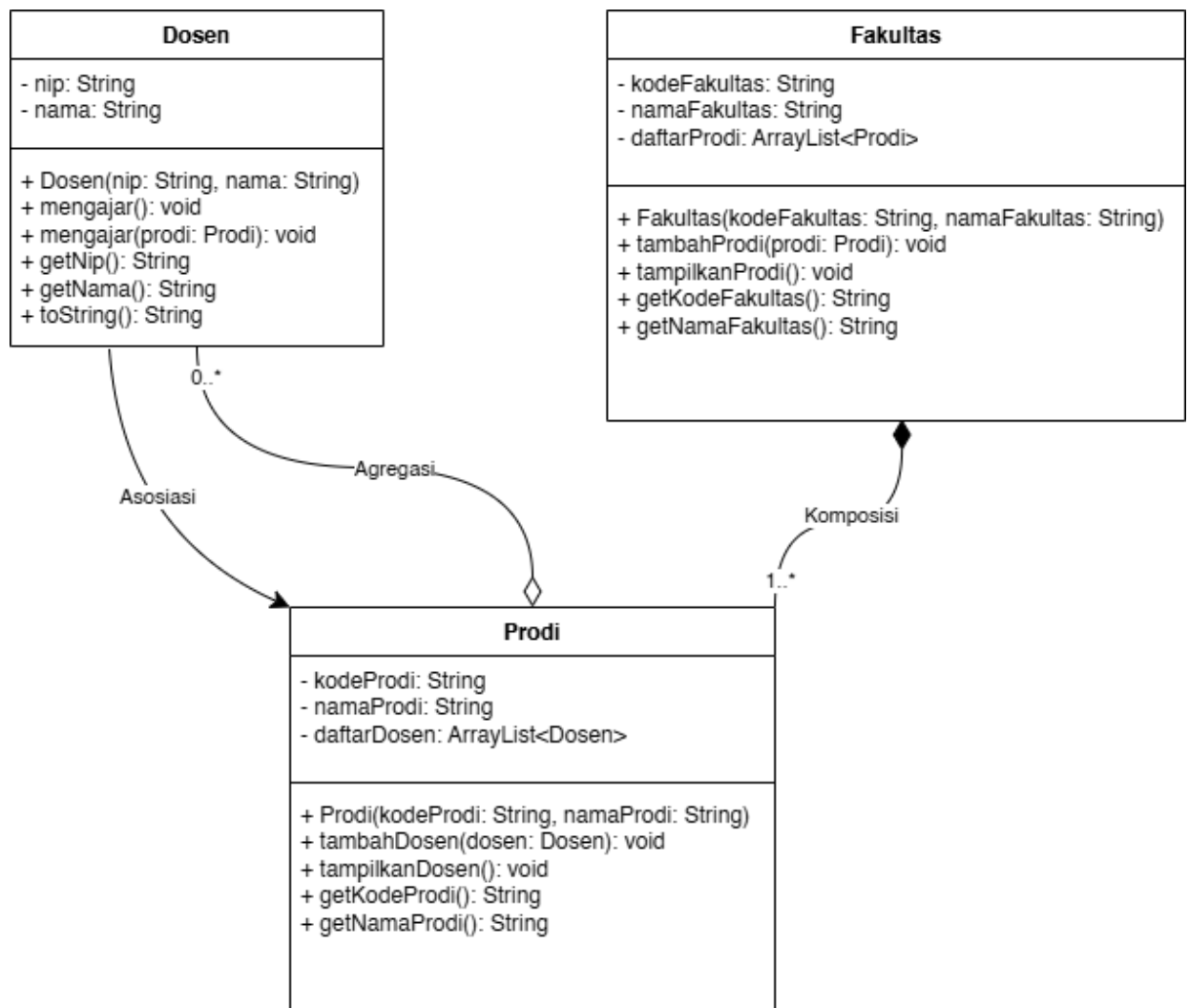
▪ Identitas Praktikum

Nama MK : Praktikum Pemrograman Berorientasi Objek
Kode MK : CAK2KAB4
Bobot SKS : 3 SKS
Tempat : L-MM, Gedung IOT, lantai 3
Hari, tanggal : Selasa, 14 April 2026
Jam : 09:30-12:30 WIB
Topik praktikum : Modul-5

▪ Identitas Mahasiswa

Nama lengkap : Keishin Naufa Alfaridzhi
NIM : 103112400061
Program Studi : S-1 Teknik Informatika

▪ Class Diagram



- **Algoritma & Penjelasan**

Program ini merupakan implementasi dari konsep **Association** dalam Pemrograman Berorientasi Objek, yang menggambarkan hubungan antar objek di lingkungan universitas. Terdapat tiga class utama yaitu Dosen, Prodi, dan Fakultas, serta satu class **Main** sebagai titik eksekusi program. Hubungan antar class bersifat *association* (has-a), di mana Fakultas memiliki banyak Prodi, dan Prodi memiliki banyak Dosen.

- **Kode Program & Penjelasan**
(Dosen.java)

```
package Modul5.Unguided;

public class Dosen {
    private String nip;
    private String nama;

    public Dosen(String nip, String nama) {
        this.nip = nip;
        this.nama = nama;
    }

    public void mengajar() {
        System.out.println(nama + " sedang mengajar.");
    }

    public void mengajar(Prodi prodi) {
        prodi.tambahDosen(this);
        System.out.println(nama + " ditambahkan ke Prodi " + prodi.getNamaProdi());
    }

    public String getNip() {
        return nip;
    }

    public String getNama() {
        return nama;
    }

    @Override
    public String toString() {
        return nip + " - " + nama;
    }
}
```

Penjelasan:

Class **Dosen** merepresentasikan entitas dosen dengan atribut dan perilakunya. Untuk penjelasan rinci terkait class ini adalah sebagai berikut:

1. **Atribut**

Terdapat dua atribut bertipe String yaitu nip dan nama, keduanya bersifat private sehingga hanya bisa diakses dari dalam class itu sendiri.

2. Constructor

Menerima dua parameter nip dan nama, kemudian masing-masing ditetapkan ke atribut class menggunakan keyword *this*.

3. Method mengajar() (tanpa parameter)

Mencetak teks ke konsol bahwa dosen yang bersangkutan sedang mengajar, dengan format: [nama] sedang mengajar.

4. Method mengajar(Prodi prodi) (dengan parameter)

Merupakan *method overloading* dari mengajar(). Menerima objek Prodi sebagai parameter, kemudian memanggil `prodi.tambahDosen(this)` untuk mendaftarkan dosen tersebut ke dalam prodi yang dimaksud, lalu mencetak konfirmasi bahwa dosen telah ditambahkan ke prodi tersebut.

5. Getter getNip() dan getNama()

Mengembalikan nilai atribut nip dan nama yang bersifat private agar bisa diakses dari luar class.

6. Method toString()

Di-*override* dari class Object untuk mengembalikan representasi string dari objek Dosen dalam format: [nip] - [nama].

(Prodi.java)

```
package Modul5.Unguided;

import java.util.ArrayList;

public class Prodi {
    private String kodeProdi;
    private String namaProdi;
    private ArrayList<Dosen> daftarDosen;

    public Prodi(String kodeProdi, String namaProdi) {
        this.kodeProdi = kodeProdi;
        this.namaProdi = namaProdi;
        this.daftarDosen = new ArrayList<>();
    }

    public void tambahDosen(Dosen dosen) {
        if (!daftarDosen.contains(dosen)) {
            daftarDosen.add(dosen);
        }
    }

    public void tampilkanDosen() {
        System.out.println("Prodi : " + namaProdi + " (" + kodeProdi + ")");
        if (daftarDosen.isEmpty()) {
            System.out.println("Belum ada dosen.");
            System.out.println();
            return;
        }
    }
}
```

```

        for (Dosen dosen : daftarDosen) {
            System.out.println(" - " + dosen);
        }
    }

    public String getKodeProdi() {
        return kodeProdi;
    }

    public String getNamaProdi() {
        return namaProdi;
    }
}

```

Penjelasan:

Class **Prodi** merepresentasikan entitas program studi yang dapat menampung kumpulan objek Dosen. Menggunakan ArrayList sebagai struktur data untuk menyimpan daftar dosen. Untuk penjelasan rinci terkait class ini adalah sebagai berikut:

1. **Atribut**

Terdapat atribut kodeProdi dan namaProdi bertipe String, serta daftarDosen bertipe ArrayList<Dosen> untuk menampung daftar dosen yang mengajar di prodi tersebut. Ketiganya bersifat private.

2. **Constructor**

Menerima parameter kodeProdi dan namaProdi, kemudian diinisialisasi ke atribut class. daftarDosen diinisialisasi sebagai ArrayList kosong menggunakan new ArrayList<>().

3. **Method tambahDosen(Dosen dosen)**

Menerima objek Dosen sebagai parameter. Sebelum menambahkan, dilakukan pengecekan dengan daftarDosen.contains(dosen) untuk memastikan dosen yang sama tidak ditambahkan dua kali (*duplicate prevention*). Jika belum ada, dosen ditambahkan ke daftarDosen.

4. **Method tampilkanDosen()**

Mencetak nama dan kode prodi ke konsol. Jika daftarDosen kosong, mencetak teks "Belum ada dosen." dan langsung return. Jika tidak kosong, melakukan iterasi menggunakan *enhanced for-loop* untuk mencetak setiap dosen dalam format yang ditentukan oleh toString() milik class Dosen.

5. **Getter getKodeProdi() dan getNamaProdi()**

Mengembalikan nilai atribut kodeProdi dan namaProdi agar dapat diakses dari luar class.

(Fakultas.java)

```

package Modul5.Unguided;

import java.util.ArrayList;

public class Fakultas {
    private String kodeFakultas;
    private String namaFakultas;
    private ArrayList<Prodi> listProdi;

    public Fakultas(String kodeFakultas, String namaFakultas) {
        this.kodeFakultas = kodeFakultas;
        this.namaFakultas = namaFakultas;
        this.listProdi = new ArrayList<>();
    }

    public void tambahProdi(Prodi prodi) {
        if (!listProdi.contains(prodi)) {
            listProdi.add(prodi);
        }
    }

    public void tampilkanProdi() {
        System.out.println("Fakultas : " + namaFakultas + " (" + kodeFakultas + ")");
        if (listProdi.isEmpty()) {
            System.out.println("Belum ada prodi.");
            return;
        }
        for (Prodi prodi : listProdi) {
            System.out.println(" - " + prodi.getKodeProdi() + " " + prodi.getNama-
Prodi());
        }
    }

    public String getKodeFakultas() {
        return kodeFakultas;
    }

    public String getNamaFakultas() {
        return namaFakultas;
    }
}

```

Penjelasan:

Class **Fakultas** merepresentasikan entitas fakultas yang dapat menampung kumpulan objek Prodi. Strukturnya serupa dengan class Prodi, namun berada satu tingkat lebih tinggi dalam hierarki. Untuk penjelasan rinci terkait class ini adalah sebagai berikut:

1. Atribut

Terdapat atribut kodeFakultas dan namaFakultas bertipe String, serta listProdi bertipe ArrayList<Prodi> untuk menampung daftar prodi yang berada di bawah fakultas tersebut. Ketiganya bersifat private.

2. Constructor

Menerima parameter kodeFakultas dan namaFakultas, kemudian diinisialisasi ke atribut class. listProdi diinisialisasi sebagai ArrayList kosong.

3. **Method tambahProdi(Prodi prodi)**

Menerima objek Prodi sebagai parameter. Melakukan pengecekan dengan listProdi.contains(prodi) sebelum menambahkan untuk mencegah duplikasi. Jika prodi belum ada dalam list, maka prodi ditambahkan ke listProdi.

4. **Method tampilkanProdi()**

Mencetak nama dan kode fakultas ke konsol. Jika listProdi kosong, mencetak "Belum ada prodi." dan berhenti. Jika tidak kosong, melakukan iterasi menggunakan *enhanced for-loop* untuk mencetak kode dan nama setiap prodi.

5. **Getter getKodeFakultas() dan getNamaFakultas()**

Mengembalikan nilai atribut kodeFakultas dan namaFakultas agar dapat diakses dari luar class.

(Main.java)

```
package Modul5.Unguided;

public class Main {
    public static void main(String[] args) {
        Fakultas fakultasInformatika = new Fakultas("FIF01", "Fakultas teknikInformatika");

        Prodi teknikInformatika = new Prodi("IF01", "Teknik Informatika");
        Prodi rpl = new Prodi("RPL01", "Rekayasa Perangkat Lunak");

        fakultasInformatika.tambahProdi(teknikInformatika);
        fakultasInformatika.tambahProdi(rpl);

        teknikInformatika.tampilkanDosen();
        rpl.tampilkanDosen();

        Dosen dosen1 = new Dosen("D001", "Sombrenion");
        Dosen dosen2 = new Dosen("D002", "Companion");
        Dosen dosen3 = new Dosen("D003", "Sonion");

        dosen1.mengajar(teknikInformatika);
        dosen2.mengajar(teknikInformatika);
        dosen3.mengajar(rpl);

        Dosen dosenBebas = new Dosen("D004", "Joc");
        dosenBebas.mengajar();

        System.out.println();
        fakultasInformatika.tampilkanProdi();
        System.out.println();
        teknikInformatika.tampilkanDosen();
        System.out.println();
        rpl.tampilkanDosen();
    }
}
```

Penjelasan:

Class **Main** merupakan titik masuk (*entry point*) dari program. Di sinilah seluruh objek dibuat dan dihubungkan satu sama lain untuk mensimulasikan struktur universitas. Untuk penjelasan rinci terkait eksekusi program adalah sebagai berikut:

1. Inisialisasi Objek Fakultas dan Prodi

Membuat satu objek Fakultas bernama fakultasInformatika dengan kode "FIF01". Kemudian membuat dua objek Prodi yaitu teknikInformatika (kode "IF01") dan rpl (kode "RPL01"). Kedua prodi tersebut didaftarkan ke dalam fakultas menggunakan fakultasInformatika.tambahProdi(...).

2. Menampilkan Dosen Awal (Kosong)

Sebelum ada dosen yang ditambahkan, tampilkanDosen() dipanggil pada kedua prodi. Karena daftarDosen masih kosong, output yang muncul adalah "Belum ada dosen." untuk masing-masing prodi.

3. Inisialisasi Objek Dosen dan Penugasan ke Prodi

Membuat tiga objek Dosen yaitu dosen1 (Sombrenion), dosen2 (Companion), dan dosen3 (Sonion). Kemudian memanggil mengajar(Prodi) pada masing-masing dosen: dosen1 dan dosen2 ditambahkan ke teknikInformatika, sedangkan dosen3 ditambahkan ke rpl.

4. Dosen Bebas

Membuat objek dosenBebas (Joc) dan memanggil mengajar() tanpa parameter, sehingga hanya mencetak "Joc sedang mengajar." tanpa mendaftarkan dosen tersebut ke prodi manapun.

5. Menampilkan Struktur Akhir

Memanggil tampilkanProdi() pada fakultasInformatika untuk menampilkan seluruh prodi yang terdaftar. Kemudian memanggil tampilkanDosen() pada teknikInformatika dan rpl untuk menampilkan daftar dosen yang sudah terdaftar di masing-masing prodi.

▪ Hasil Running Program

```
Prodi : Teknik Informatika (IF01)
Belum ada dosen.

Prodi : Rekayasa Perangkat Lunak (RPL01)
Belum ada dosen.

Sombrenion ditambahkan ke Prodi Teknik Informatika
Companion ditambahkan ke Prodi Teknik Informatika
Sonion ditambahkan ke Prodi Rekayasa Perangkat Lunak
Joc sedang mengajar.

Fakultas : Fakultas teknikInformatika (FIF01)
- IF01 Teknik Informatika
- RPL01 Rekayasa Perangkat Lunak

Prodi : Teknik Informatika (IF01)
- D001 - Sombrenion
- D002 - Companion

Prodi : Rekayasa Perangkat Lunak (RPL01)
- D003 - Sonion
```

▪ Link Program Praktikum (Github)

[kukingkux/PRAKTIKUM-PBO](https://github.com/kukingkux/PRAKTIKUM-PBO)